PitForge: Exact Ultimate-Pit Optimization Reproducing the Published MineLib Optima, with Whittle Nested Shells and Scheduling

Felipe Santibañez-Leal

***Abstract*—The first optimization of open-pit mine design is the ultimate pit limit: given a block model in which each block carries a net economic value and a set of slope-precedence constraints, find the set of blocks to extract that maximises total value while respecting the slopes. The problem has an exact polynomial solution as the maximum-weight closure of the precedence graph, computed as a minimum cut, and PitForge implements it to run *live in the browser*: drag the price, revenue factor or slope and the exact pit re-solves instantly. This report validates the solver against ground truth and extends it to the two decisions that follow. The validation is the strongest kind available: on the three public MineLib instances (newman1, zuck_small and kd, 1000 to 14000 blocks), the solver reproduces the *published* optimum to a relative error near 10^{-10}, an exact match, and solves the largest in about a quarter of a second in a browser tab. Beyond the single pit, the tool computes the Whittle nested pit shells, the family of optimal pits over an ascending revenue-factor schedule, with their value, tonnage and strip-ratio curves that guide phase and pushback design, and it solves a constrained production schedule over time, reporting both a certified upper-bound net present value and the achievable rounded schedule, and honestly, the 11% optimality gap between them. Every pit value is self-checked (pit value $=$ $\sum$ positive value $-$ max flow is asserted on every solve), the MineLib instances are fetched per their academic-download terms with only summary numbers committed, and every figure regenerates from the committed artifacts.**

***Index Terms*—Open-pit mining, ultimate pit limit, maximum closure, minimum cut, nested pit shells, Whittle, production scheduling, MineLib, reproducibility.**

I. Introduction

THE ultimate pit limit (UPL) is the foundational optimization of open-pit mine design: given a three-dimensional block model in which each block has a net value (recoverable metal less mining and processing cost) and slope-precedence constraints (a block can be mined only after those above it in the pit wall), which set of blocks maximises total value? The problem has an exact, polynomial-time solution, established by Lerchs and Grossmann [1] and understood through Picard's theorem [2] as the maximum-weight closure of the precedence graph, which is the source side of a minimum s–t cut and hence solvable by any maximum-flow algorithm [3], [4]. This is textbook operations research [8], but three things turn the textbook solve into a useful tool: it must be validated against known answers, it must extend to the family of nested pits used for phase design, and it must connect to the production schedule.

F. Santibañez-Leal is with the CAOS open-research programme, Santiago, Chile (e-mail: fsantibanez@gmail.com; ORCID 0000-0002-0150-3246).

Technical report, 2026. Interactive tool and reproducible pipeline (MIT) at https://github.com/fsantibanezleal/CAOS_PitForge; live at https://pitforge.fasl-work.com.

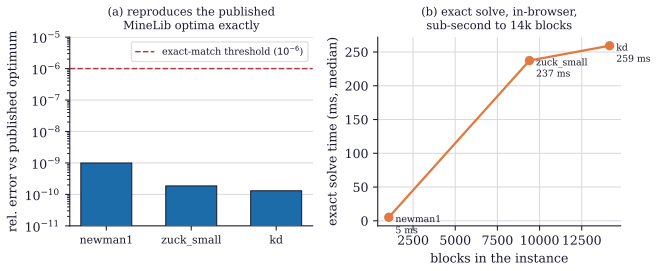

Fig. 1. Validation on the public MineLib instances. **(a)** The solver reproduces the *published* optimum of newman1, zuck_small and kd to a relative error near 10^{-10}, far below any exact-match threshold. **(b)** The exact solve is fast enough to run interactively in a browser: about 5 ms for the 1060-block newman1, and about a quarter of a second for the 14153-block kd.

PitForge does all three, exactly and in the browser. It solves the UPL as a minimum cut, self-checking the value identity on every solve, and it re-solves instantly as a user drags the economic parameters. This report validates it against the published MineLib benchmark optima [6], the gold standard for a pit optimiser, and reports the two extensions: the Whittle nested pit shells [5], the optimal pits across an ascending revenue factor that give the value-tonnage-strip curves phase design rests on, and a constrained production schedule [7] with its honest optimality gap. The contribution is not a new algorithm; it is an exact, validated, transparent implementation that reproduces the literature and exposes the honest gaps where they remain.

II. The Exact Pit, Validated on MineLib

The ultimate pit is the maximum-weight closed set of the precedence graph, and PitForge computes it by Picard's reduction to a minimum cut: a source arc to every positive-value block, a sink arc from every negative block, and an infinite-capacity arc for each precedence relation, after which a Dinic maximum flow gives the pit as the source-reachable set, with the value identity pit value $= \sum_{v_b>0} v_b -$ maxflow asserted on every solve so a wrong answer cannot be returned silently. The validation is against the strongest available reference (Fig. 1, Table I): the three public MineLib instances, whose optima are published. On all three, PitForge's value matches the published optimum to a relative error near 10^{-10}, 9.96×10^{-10} on newman1, 1.86×10^{-10} on zuck_small, 1.30×10^{-10} on kd, an

TABLE I
PITFORGE VERSUS THE PUBLISHED MINELIB OPTIMA. THE SOLVER MATCHES ALL THREE TO RELATIVE ERROR NEAR 10^{-10}, IN MILLISECONDS TO A FRACTION OF A SECOND (IN-BROWSER, TYPESCRIPT DINIC MIN-CUT).

Instance	blocks	published optimum	rel. error	solve (ms)
newman1	1060	2.6087×10^7	1.0×10^{-9}	5
zuck_small	9400	1.4227×10^9	1.9×10^{-10}	237
kd	14153	6.5220×10^8	1.3×10^{-10}	259

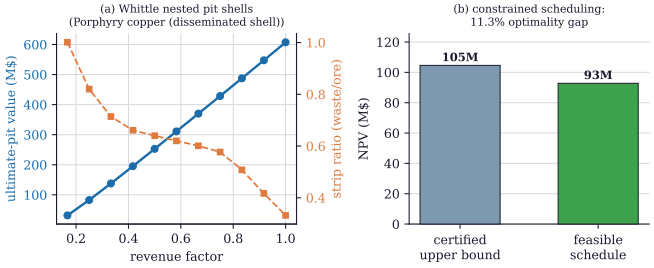

Fig. 2. Beyond the single pit. **(a)** The Whittle nested pit shells: solving the ultimate pit over an ascending revenue factor gives a family of nested pits whose value rises and whose strip ratio falls as the pit grows, the value-tonnage-strip curves that phase and pushback design read. **(b)** A constrained production schedule over eight discounted periods: the certified upper-bound net present value (105M) and the achievable rounded schedule (93M), whose 11.3% gap is the honest cost of a feasible integer schedule against the relaxation bound.

exact match to floating-point precision. And it is fast: the 1060-block newman1 solves in about 5 milliseconds, and the 14153-block kd in about 259 milliseconds, both inside a browser tab, so the exact pit re-solves interactively as a user changes the economics. Reproducing the published optima exactly, in the browser, is the validation a pit optimiser needs, and it is the foundation the rest of the tool builds on.

III. NESTED SHELLS AND SCHEDULING

A mine is not designed as one pit but as a sequence, and PitForge computes the two objects that sequence rests on (Fig. 2). The first is the Whittle nested pit shells [5]: solving the ultimate pit over an ascending schedule of revenue factors (multiplying the metal price) yields a family of nested optimal pits, from a small high-grade core at a low revenue factor to the full pit at a factor of one, and their value, tonnage and strip-ratio curves (Fig. 2a) are exactly what a planner uses to choose phases and pushbacks. Because each shell is an exact ultimate-pit solve, the whole family is exact. The second is the production schedule: assigning the pit's blocks to time periods under a capacity constraint to maximise discounted value is the constrained pit scheduling problem [7], harder than the UPL, and PitForge reports it honestly (Fig. 2b), a certified upper-bound net present value of 105 million from the relaxation, and an achievable rounded schedule of 93 million, with the 11.3% gap between them stated rather than hidden. That gap is the real difficulty of scheduling: the ultimate pit is solved exactly, but turning it into a feasible, capacity-limited, time-phased plan leaves an optimality gap that an honest tool reports.

IV. DISCUSSION AND SCOPE

PitForge's value is an exact, validated, transparent pit optimiser that does the whole first stage of open-pit design in the browser and is honest about where exactness ends. The ultimate pit and the nested shells are solved *exactly*, and the MineLib reproduction to relative error 10^{-10} is the proof; the schedule is solved to a certified bound with the optimality gap reported, because the scheduling problem, unlike the UPL, does not admit a cheap exact solve at these sizes. The self-checking value identity means a returned pit value is guaranteed consistent, not merely plausible. The scope is stated plainly. The MineLib instances are fetched at runtime under their academic-download grant and are not re-hosted; only the summary numbers (the optima and timings) are committed. The synthetic cases used for the nested shells and the schedule are physically reasonable block models, not a specific deposit, so their numbers illustrate the method rather than predict a mine. And the tool computes the pit and the schedule; it does not replace the geological, geotechnical and financial judgement that turns them into a mine plan. None of this is hidden; the interface asserts the value identity and states the scheduling gap on every solve.

V. CONCLUSION

PitForge solves the open-pit ultimate-pit limit exactly, as a maximum-closure minimum cut, live in the browser, and reproduces the three published MineLib optima to relative error near 10^{-10}, in milliseconds to a quarter of a second. It extends the exact solve to the Whittle nested pit shells that guide phase design, and to a constrained production schedule whose 11% optimality gap it reports rather than conceals. Reproducing the benchmark optima exactly, self-checking every solve, and stating the scheduling gap honestly are the report's discipline, and the transparent, reproducible optimiser that does it is its contribution.

APPENDIX A
REPRODUCTION

The committed artifacts under `data/derived/` carry the MineLib benchmark (`minelib-results.json`: per-instance published optimum, solved value, relative error and timing), the nested-shell curves per case (`case-results.json`: revenue factor, pit value, tonnage and strip ratio), and the constrained schedule (`cpit-schedule.json`: the certified bound and rounded-schedule net present values). The figure scripts under `manuscripts/ultimate-pit/figures/` read those artifacts; the exact solver is deterministic, so re-running reproduces Table I and Figs. 1–2. The MineLib instances themselves are fetched from public mirrors under their academic-download terms and are not committed.

REFERENCES

[1] H. Lerchs and I. F. Grossmann, "Optimum design of open-pit mines," *Transactions of the Canadian Institute of Mining and Metallurgy*, vol. 68, pp. 17–24, 1965.

[2] J.-C. Picard, "Maximal closure of a graph and applications to combinatorial problems," *Management Science*, vol. 22, no. 11, pp. 1268–1272, 1976. doi:10.1287/mnsc.22.11.1268.

[3] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. doi:10.4153/CJM-1956-045-5.

[4] D. S. Hochbaum, "The pseudoflow algorithm: A new algorithm for the maximum-flow problem," *Operations Research*, vol. 56, no. 4, pp. 992–1009, 2008. doi:10.1287/opre.1080.0524.

[5] J. Whittle, "Beyond optimization in open pit design," in *Proc. Canadian Conf. on Computer Applications in the Mineral Industry*, Rotterdam: Balkema, 1988, pp. 331–337.

[6] D. Espinoza, M. Goycoolea, E. Moreno, and A. Newman, "MineLib: A library of open pit mining problems," *Annals of Operations Research*, vol. 206, no. 1, pp. 93–114, 2013. doi:10.1007/s10479-012-1258-3.

[7] R. Chicoisne, D. Espinoza, M. Goycoolea, E. Moreno, and E. Rubio, "A new algorithm for the open-pit mine production scheduling problem," *Operations Research*, vol. 60, no. 3, pp. 517–528, 2012. doi:10.1287/opre.1120.1050.

[8] A. M. Newman, E. Rubio, R. Caro, A. Weintraub, and K. Eurek, "A review of operations research in mine planning," *Interfaces*, vol. 40, no. 3, pp. 222–245, 2010. doi:10.1287/inte.1090.0492.